

Indicații

- Testul conține 3 subiecte, durează 120 de minute și valorează 50 de puncte, cu alte 5 bonus.
- Se pot obține punctaje parțiale pentru un subiect numai dacă acest lucru este specificat.
- Pentru a fi punctată, o rezolvare **trebuie** să includă și metoda de verificare a funcționalității acesteia.
- Pentru subiectul 1, pentru afișarea la ecran este recomandată utilizarea macroului `PRINTF32`.
- Pentru subiectul 2, pentru afișarea la ecran este obligatorie utilizarea funcției de bibliotecă `printf()`.
- Ordinea de rezolvare a subiectelor este la alegerea voastră.
- Subiectele se rezolvă pe mașina virtuală de PCLP2.

Subiecte

Subiectul 1

Studentii pasionați de IOCLA sunt pasionați și de numerologie. Demonstrați-vă apartenența la această categorie.

- [5p] Numim un număr perfect, dacă cei 2 biți centrali ai săi sunt egali. Verificați dacă un număr pe 8 biți este perfect. Pentru testare, folosiți variabila `perfect` salvată în secțiunea `.data`. Dacă aceasta este perfectă, afișați mesajul “The number is perfect”, altfel, “The number is not perfect”.
- [5p] Numim un șir de numere pur, dacă toate elementele sale de pe poziții puteri ale lui 2 sunt perfecte. Verificați dacă un șir este pur, folosind pentru testare vectorul `pure` salvat în secțiunea `.data`. Dacă acesta este pur, afișați mesajul “The array is pure”, altfel, “The array is not pure”.
- [5p] Numim o adresă magică, dacă deferențiindu-o de 3 ori obținem valoarea 0. Verificați dacă o adresă este magică, folosind pentru testare variabila `address` salvată în secțiunea `.data`. Dacă aceasta este magică, afișați mesajul “The address is magic”, altfel, “The address is not magic”.
- [5p] Pentru un vector de întregi ce conține duplicate și o singură valoare unică, determinați, **folosind o singură buclă**, elementul neduplicat. Folosiți pentru testare vectorul `duplicates` salvat în secțiunea `.data` și afișați valoarea fără duplicate din acesta.

Subiectul 2

O variantă eficientă și performantă de a reține mulțimile de numere naturale este folosirea bitseturilor.

Un bitset este un vector de octeți care funcționează similar cu un vector frecvență, însă reține apartenența unui element la mulțime prin setarea **bitului** corespunzător. Spre exemplu, pentru a marca faptul că valoarea 56 aparține unei mulțimi, trebuie setat cel de-al 56-lea bit din vectorul bitsetului.

Bitsetul este declarat în secțiunea `.bss` cu numele `bitset` și are dimensiunea maximă de 1000 de octeți.

- [5p] Implementați funcția `void insert_in_set(char *bitset, int n, int x)` care introduce elementul `x` în mulțimea reținută în bitsetul `bitset` de dimensiune `n` octeți. Pentru testare, introduceți valoarea 56 în bitsetul gol și afișați valoarea octetului în care aceasta a fost salvată.
- [5p] Implementați funcția `void read_set(char *bitset, int n, int m)` care citește `m` elemente de la tastatură și le introduce în mulțimea reținută în bitsetul `bitset` de dimensiune `n` octeți. Pentru testare, citiți de la tastatură valorile 160 și 161 și afișați valoarea octetului din bitset în care acestea au fost salvate.
- [5p] Implementați funcția `void print_set(char *bitset, int n)` care afișează elementele mulțimii reținute în bitsetul `bitset` de dimensiune `n` octeți. Pentru testare, folosiți bitsetul populat la punctele anterioare.
- [5p] Implementați funcția `int card_set(char *bitset, int n)` care afișează cardinalul mulțimii reținute în bitsetul `bitset` de dimensiune `n` octeți. Pentru testare, folosiți bitsetul populat la punctele anterioare.

Subiectul 3

- [5p] Adăugați o regulă în `Makefile` pentru a schimba entry-point-ul executabilului rezultat din fișierul obiect `entry.o` din funcția `main` în funcția `entry`. Ambele funcții sunt definite deja în `entry.o`.
- [5p] Inspectați executabilul `attack` și găsiți o modalitate pentru a declanșa afișarea mesajului “You got the key!”.

- c) [5p] Inspectați executabilul **variables** și găsiți o modalitate pentru a declanșa afișarea mesajului “You populated the right variable. Nice!”.